

To pop(delete) the elements from the list

```
lst=[1,2,3,4,5]  
lst.pop()  
print(lst)
```

```
[1, 2, 3, 4]
```

```
lst=['one','two','three','four','five']  
lst.pop(1)  
print(lst)
```

```
['one', 'three', 'four', 'five']
```

Delete

```
lst=['one','two','three','four','five']  
del lst[0]  
print(lst)
```

```
['two', 'three', 'four', 'five']
```

```
marks=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]  
del marks[0:10]  
marks
```

```
[11, 12, 13, 14, 15]
```

Reverse

```
lst=['one','two','three','four','five']  
lst.reverse()  
print(lst)
```

```
['five', 'four', 'three', 'two', 'one']
```

Sorting

```
numbers=[3,1,6,2,8]  
print("Original list:",numbers)  
sorted_lst=sorted(numbers)  
print("Sorted list:",sorted_lst)  
print("Reverse Sorted list:",sorted(numbers,reverse=True))
```

```
Original list: [3, 1, 6, 2, 8]  
Sorted list: [1, 2, 3, 6, 8]  
Reverse Sorted list: [8, 6, 3, 2, 1]
```

```
#sort is applicable for only numeric values  
lst=[1,20,'b',5,'a']  
print(lst.sort())
```

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_7217/4258688497.py in <cell line: 0>()  
      1 #sort is applicable for only numeric values  
      2 lst=[1,20,'b',5,'a']  
----> 3 print(lst.sort())  
  
TypeError: '<' not supported between instances of 'str' and 'int'
```

Count

```
numbers=[1,2,3,1,1,0,4,2,5]  
print(numbers.count(1))#frequency of 1 in the list
```

```
print(numbers.count(3))#frequency of 3 in the list
```

List Looping--->List Iteration

```
#accessing list elements
marks=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
for mark in marks:
    print(mark)
```

List Comprehension

```
#without list comprehension
squares=[]
for i in range(10):
    squares.append(i**2)
print(squares)
```

```
#using list comprehension
squares=[i**2 for i in range(10)]
print(squares)
```

```
print([i**2 for i in range(10)])
```

Tuple creation

```
tup1=()
type(tup1)
```

```
tuple
```

```
tup2=tuple()
type(tup2)
```

```
tuple
```

```
tup3=(1,)  
type(tup3)
```

```
tuple
```

```
tup3=(100,'shiva',17.769)  
tup3
```

```
(100, 'shiva', 17.769)
```

```
tup4=('shiva',25,(50,100),(150,90))
```

```
tup5=('shiva',25,(50,100),(150,90))
```

```
len(tup5)
```

```
4
```

Tuple Indexing

```
tup5[0]#Retrieve the first element of tuple
```

```
'shiva'
```

```
tup5[2][0]
```

```
50
```

```
tup4[-1]
```

```
(150, 90)
```

Tuple Slicing

```
mytuple=('one','two','three','four','five','six','seven','eight','nine','ten')
```

```
#Returns elements from 0 to 3  
mytuple[0:3]
```

```
('one', 'two', 'three')
```

```
#returns  
mytuple[2:5]
```

```
('three', 'four', 'five')
```

```
mytuple[:3]#return first three elements
```

```
('one', 'two', 'three')
```

Remove and change the item

```
mytuple
```

```
('one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten')
```

```
del mytuple[0]
```

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_7217/1004790074.py in <cell line: 0>()  
----> 1 del mytuple[0]
```

```
TypeError: 'tuple' object doesn't support item deletion
```

```
mytuple[0]=1
```

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_7217/1923060558.py in <cell line: 0>()  
----> 1 mytuple[0]=1  
  
TypeError: 'tuple' object does not support item assignment
```

```
del mytuple #deleting entire tuple is possible
```

Loop through tuple

```
mytuple=('one','two','three','four','five','six','seven','eight','nine','ten')
```

```
for i in mytuple:  
    print(i)
```

Inbuilt Functions

```
mytuple=('one','two','three','one','one','six','seven','eight','nine','ten')
```

```
mytuple.count("one")
```

Index position

```
mytuple.index('one')#index of first element
```

```
mytuple.index('seven')#index of seven element
```

Sorting

```
mytuple=(43,67,99,12,6,90,67)
```

```
sorted(mytuple)
```

```
sorted(mytuple, reverse='true')
```

Tuple membership

```
mytuple=('one','two','three','four','five','six','seven','eight','nine','ten')
```

```
'one' in mytuple
```

Sets

```
set_var=set()  
print(set_var)  
print(type(set_var))
```

```
set()  
<class 'set'>
```

union, intersection,difference,symmetric difference

```
a={10,20,30,40,'a'}  
print(a)
```

```
{20, 40, 10, 30, 'a'}
```

```
#automatically set won't consider duplicate values
s={10,20,20,30,30,40,40}
print(s)
```

```
{40, 10, 20, 30}
```

```
print(s[1]) #set does not support indexing
```

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_1009/2477845534.py in <cell line: 0>()
----> 1 print(s[1]) #set does not support indexing

TypeError: 'set' object is not subscriptable
```

```
c=set()
c
```

```
set()
```

Inbuilt Functions

ADD()

```
c.add(3)
c
```

```
{3}
```

```
c.add(4)
c.add(5)
c.add(6)
c.add('a')
c.add('true')
```

```
c.add(0)
c
```

```
{0, 3, 4, 5, 6, 'a', 'true'}
```

Update() Add Multiple Elements

```
s=set()
s.update([5,6,1])
print(s)
```

```
{1, 5, 6}
```

```
s.update([8,9],[10,3,2])
print(s)
```

```
{1, 2, 3, 5, 6, 8, 9, 10}
```

Delete elements from set

- Discard()
- Remove()

```
s={1,2,3,4,5}
print(s)
s.discard(4)
print(s)
```

```
{1, 2, 3, 4, 5}
{1, 2, 3, 5}
```

```
s.discard(7)
print(s)
```

```
{1, 2, 3, 5}
```

```
s.remove(1)  
print(s)
```

```
{2, 3, 5}
```

```
s.remove(7)  
print(s)
```

```
-----  
KeyError                                Traceback (most recent call last)  
/tmp/ipykernel_1009/902906152.py in <cell line: 0>()  
----> 1 s.remove(7)  
      2 print(s)  
  
KeyError: 7
```

```
s={2,2,3,7,4,"Shiva","Ram"}  
s.pop()  
print(s)
```

```
{3, 4, 'Ram', 7, 'Shiva'}
```

```
s={1,5,2,3,6}  
s.clear()  
print(s)
```

```
set()
```

```
s={1,2,3,4,5}  
print(s)
```

```
{1, 2, 3, 4, 5}
```

```
del(s)
```

Set Operations

Union

```
set1={1,2,3,4,5}  
set2={3,4,5,6,7}  
print(set1|set2)
```

```
{1, 2, 3, 4, 5, 6, 7}
```

```
print(set1.union(set2))
```

```
{1, 2, 3, 4, 5, 6, 7}
```

Intersection

```
print(set1 & set2)
```

```
{3, 4, 5}
```

Difference

```
set1={1,2,3,4,5}  
set2={3,4,5,6,7}  
print(set1-set2)
```

```
{1, 2}
```

```
set1={1,2,3,4,5}
set2={3,4,5,6,7}
print(set2-set1)
```

```
{6, 7}
```

```
print(set1.symmetric_difference(set2))
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_595/345630970.py in <cell line: 0>()
----> 1 print(set1.symmetric_difference(set2))

NameError: name 'set1' is not defined
```

issubset()

```
c={2,3,4,1,5,6,7,8}
d={2,1,5,8}
d.issubset(c)
```

```
True
```

issuperset()

```
d.issuperset(c)
```

```
-----
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_595/489653298.py in <cell line: 0>()
----> 1 d.issuperset(c)

AttributeError: 'dict' object has no attribute 'issuperset'
```

Dictionary

```
dic={}
type(dic)
```

```
dict
```

```
d=dict()
type(d)
```

```
dict
```

```
my_dict={"Car1": "Audi","Car2": "BMW","Car3":"Mercedez"}
type(my_dict)
print(my_dict)
```

```
{'Car1': 'Audi', 'Car2': 'BMW', 'Car3': 'Mercedez'}
```

```
print(len(my_dict))
```

```
3
```

```
dict1={"key1":456,"key2":[3,9,15],"key3):(94,8,23),"key4":["item1','item2','item3']}
dict1['key2']
```

```
[3, 9, 15]
```

```
dict1.keys()
```

```
dict_keys(['key1', 'key2', 'key3', 'key4'])
```

```
dict1.values()
```

```
dict_values([456, [3, 9, 15], (94, 8, 23), ['item1', 'item2', 'item3']])
```

```
dict1.items()#prints the whole dictionary
```

```
dict_items([('key1', 456), ('key2', [3, 9, 15]), ('key3', (94, 8, 23)), ('key4', ['item1', 'item2', 'item3'])])
```

```
my_dict["Car5"]="Hyundai"  
my_dict
```

```
{'Car1': 'Maruti',  
 'Car2': 'BMW',  
 'Car3': 'Mercedez',  
 'car4': 'Audi 2.0',  
 'Car4': 'Hyundai',  
 'Car5': 'Hyundai'}
```

```
my_dict["Car1"]="Maruti"
```

```
my_dict
```

```
{'Car1': 'Maruti',  
 'Car2': 'BMW',  
 'Car3': 'Mercedez',  
 'car4': 'Audi 2.0',  
 'Car4': 'Hyundai',  
 'Car5': 'Hyundai'}
```

Nested Dictionary

```
car1_model={'Mercedes':1960}  
car2_model={'Audi':1970}  
car3_model={'Ambassador':1980}  
car_type={'car1':car1_model,'car2':car2_model,'car3':car3_model}
```

```
print(car_type)
```

```
{'car1': {'Mercedes': 1960}, 'car2': {'Audi': 1970}, 'car3': {'Ambassador': 1980}}
```

```
thisdict={  
    "brand":"Ford",  
    "model":"Mustang",  
    "year":1964  
}  
thisdict.update({"colour":"red"})  
print(thisdict)
```

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'colour': 'red'}
```

```
numbers=map(int,input("Enter number separated by space ").split())#map applies int to every element #splits th  
print(list(numbers))
```

```
Enter number separated by space 10 20 30 40  
[10, 20, 30, 40]
```